

2.3. Funciones y programación orientada a objetos en Dart



DESARROLLO DE APLICACIONES WEB MULTIPLATAFORMA NIVEL 3.

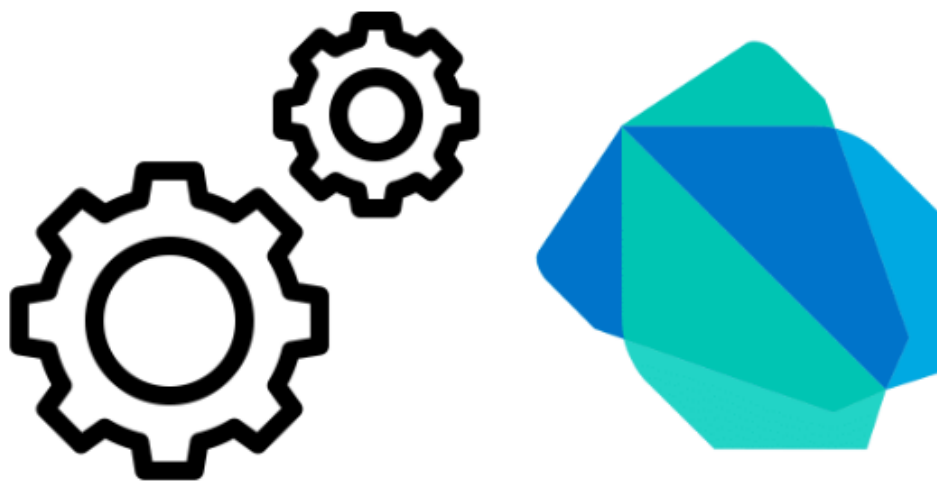
UNIDAD 2:

FUNDAMENTOS DE DART

Contenido

Funciones en Dart	3
Tipos de funciones	3
Cuando usar cada tipo de función:.....	4
Definición y sintaxis de funciones	4
Sintaxis tradicional:	4
Sintaxis flecha (Dart 2.0 y superior):	4
Parámetros y argumentos.....	5
Diferencias en la definición de parámetros según los dos tipos de sintaxis:	5
Flexibilidad en los nombres de parámetros:	6
Parámetros opcionales y por defecto:	7
Ámbito de las variables de parámetros:	8
Funciones con o sin retorno de valor	8
Funciones con retorno de valor:	8
Funciones sin retorno de valor:	9
Ámbito de las variables en las funciones	10
Aplicación en ambas sintaxis:	11
Programación Orientada a Objetos (POO) en Dart	12
Principios fundamentales de la Programación Orientada a Objetos (POO)	12
Conceptos fundamentales de la Programación Orientada a Objetos (POO).....	14
Diferencias entre la POO en Dart y la POO en general.....	15
Ejemplo de creación de clase, objeto, atributos y métodos en Dart	16
Creación de la clase.....	16
Creación de un objeto a partir de la clase:	19
Recursos adicionales.....	20

Funciones en Dart



Las funciones son bloques de código reutilizables que permiten realizar tareas específicas dentro de un programa. Son fundamentales para modularizar el código, hacerlo más organizado y legible, y evitar la duplicación de código.

En el desarrollo web multiplataforma con Dart, las funciones son esenciales para crear interfaces de usuario dinámicas, manejar datos y eventos, y estructurar el código de manera eficiente.

Tipos de funciones

Hay dos tipos de funciones en Dart:

- Funciones con flecha (Arrow Functions): Sintaxis concisa para funciones simples que no devuelven un valor explícito. Ejemplo: `function nombreFuncion(parámetros) => { cuerpo_función }.`
- Funciones sin flecha (Tradicionales): Sintaxis que incluye la palabra clave `return` para especificar el valor devuelto. Ejemplo: `function nombreFuncion(parámetros) { return cuerpo_función; }.`

Las funciones flecha (con sintaxis `=>`) se introdujeron en la versión 2.0 del lenguaje Dart. En las versiones anteriores (Dart 1.x) solo se podían definir funciones utilizando la sintaxis tradicional, la cual incluía la palabra clave `return` para especificar el valor devuelto.

Cuando usar cada tipo de función:

En resumen:

- La sintaxis tradicional es más adecuada para funciones complejas que devuelven valores o realizan múltiples acciones.
- La sintaxis flecha es ideal para funciones simples que no devuelven un valor explícito y solo realizan una acción

Definición y sintaxis de funciones

Las funciones se definen utilizando dos sintaxis principales: la tradicional y la sintaxis flecha, introducida en Dart 2.0.

Sintaxis tradicional:

- Utiliza la palabra clave `function` o `def` seguida del nombre de la función.
- Incluye paréntesis para los parámetros (si los hay).
- Utiliza llaves `{}` para definir el cuerpo de la función.
- Requiere la palabra clave `return` para especificar el valor que la función devuelve.

Ejemplo:

```
function calcularSuma(numero1, numero2) {  
    return numero1 + numero2;  
}
```

```
var resultadoSuma = calcularSuma(10, 5);  
print("La suma es: $resultadoSuma"); // Imprime "La suma es: 15"
```

Sintaxis flecha (Dart 2.0 y superior):

- Utiliza la palabra clave `function` o `def` seguida del nombre de la función.
- Incluye paréntesis para los parámetros (si los hay).
- Utiliza una flecha `=>` para separar los parámetros del cuerpo de la función.

- No requiere la palabra clave return si solo se realiza una acción y no se devuelve un valor explícito.

Ejemplo:

```
function saludar(nombre) => {  
    print("¡Hola, $nombre!");  
}
```

```
saludar("Pedro"); // Imprime "¡Hola, Pedro!"
```

Recomendaciones:

- Elegir la sintaxis adecuada en función de la complejidad y el propósito de la función.
- Priorizar la legibilidad y la claridad del código, independientemente de la sintaxis utilizada.
- Familiarizarse con ambas sintaxis para mayor flexibilidad en el desarrollo en Dart.

Parámetros y argumentos

En Dart, las funciones pueden recibir valores de entrada mediante parámetros, los cuales son variables definidas dentro de la función.

Cuando se llama a la función, se le pasan valores reales, llamados argumentos, que se asignan a los parámetros correspondientes.

Diferencias en la definición de parámetros según los dos tipos de sintaxis:

- Sintaxis tradicional: Los parámetros se definen dentro de los paréntesis de la función, separados por comas. Ejemplo: `function sumar(numero1, numero2) {...}`
- Sintaxis flecha: Los parámetros se definen dentro de los paréntesis de la función, separados por comas. Ejemplo: `function sumar(numero1, numero2) => {...}`

Ejemplos:

- Sintaxis flecha:

```
function sumar(numero1, numero2) => numero1 + numero2;
```

```
var resultado = sumar(5, 3); // resultado = 8
```

- Sintaxis tradicional:

```
function sumarTradicional(numero1, numero2) {  
    return numero1 + numero2;  
}
```

```
var resultadoSumaTradicional = sumarTradicional(10, 15);
```

```
print("Resultado suma tradicional:  
$resultadoSumaTradicional"); // Imprime "Resultado suma  
tradicional: 25"
```

Flexibilidad en los nombres de parámetros:

Los nombres de los parámetros no tienen que ser idénticos a los nombres de las variables que reciben los argumentos. La asignación se realiza por orden de posición, no por nombre.

```
function saludar(persona, saludo) {  
    print("$saludo, $persona!");  
}
```

```
saludar("Pedro", "Buenos días"); // Imprime "Buenos días,  
Pedro!"
```

Parámetros opcionales y por defecto:

- Parámetros opcionales: Se pueden definir parámetros que no son obligatorios al llamar a la función. Si no se proporciona un valor, se utiliza un valor por defecto o se realiza una acción específica.

Ejemplo:

```
function saludarOpcional(nombre, saludo = "Hola") {  
    print("$saludo, $nombre!");  
}
```

```
saludarOpcional("Pedro"); // Imprime "Hola, Pedro!"
```

```
saludarOpcional("Ana", "Buenos días"); // Imprime "Buenos días,  
Ana!"
```

- Parámetros por defecto: Se asignan valores predeterminados a los parámetros si no se especifican al llamar a la función.

Ejemplo:

```
function calcularPotencia(base, exponente = 2) {  
    var resultado = 1;  
    for (var i = 0; i < exponente; i++) {  
        resultado *= base;  
    }  
    return resultado;  
}
```

```
var potenciaCuadrado = calcularPotencia(5); // Imprime 25 (5  
elevado a 2)
```

```
var potenciaCubo = calcularPotencia(5, 3); // Imprime 125 (5  
elevado a 3)
```

Ámbito de las variables de parámetros:

Las variables declaradas como parámetros solo son accesibles dentro del cuerpo de la función donde se definen.

No se pueden modificar variables fuera de la función desde los parámetros.

Funciones con o sin retorno de valor

En Dart las funciones pueden definirse con o sin retorno de valor, y ambas opciones pueden expresarse en cualquiera de las dos sintaxis: tradicional y flecha.

Funciones con retorno de valor:

Las funciones con retorno de valor devuelven un resultado o un dato específico tras su ejecución.

Sintaxis:

- Tradicional:

```
function nombreFuncion(parámetros) {  
    return valorDevuelto;  
}
```

- Flecha:

```
function nombreFuncion(parámetros) => valorDevuelto;
```

Ejemplo:

```
function sumar(numero1, numero2) => numero1 + numero2;
```

```
var resultadoSuma = sumar(5, 3); // resultadoSuma = 8
```


Funciones sin retorno de valor:

Las funciones sin retorno de valor no devuelven un resultado específico, sino que realizan una acción o modifican datos.

Sintaxis:

- Tradicional:

```
function nombreFuncion(parámetros) {  
    // Instrucciones para realizar la acción  
}
```

- Flecha:

```
function nombreFuncion(parámetros) => {  
    // Instrucciones para realizar la acción  
};
```

Ejemplo:

```
function saludar(nombre) {  
    print("¡Hola, $nombre!");  
}
```

```
saludar("Pedro"); // Imprime "¡Hola, Pedro!"
```

Ámbito de las variables en las funciones

El ámbito de las variables en Dart es un concepto fundamental que define la accesibilidad de las variables dentro de un programa.

Las variables declaradas dentro de una función solo son accesibles dentro de la misma función. Son variables locales. Las variables declaradas fuera de las funciones son accesibles desde cualquier lugar del programa, excepto dentro de otras funciones que tienen un ámbito local propio. Son variables globales

Ejemplo con variable global y variable local:

```
var variableGlobal = "Valor global";

function mostrarVariable() {
  var variableLocal = "Valor local";
  print(variableLocal); // Imprime "Valor local"
  print(variableGlobal); // Imprime "Valor global"
}

mostrarVariable();

// print(variableLocal); // Error: variableLocal no está definida
// fuera de la función

Usa el código con precaución.
```

Explicación:

- La variable `variableGlobal` se declara fuera de la función, por lo que tiene un ámbito global.
- La variable `variableLocal` se declara dentro de la función `mostrarVariable`, por lo que tiene un ámbito local.
- Dentro de la función, se puede acceder a ambas variables: `variableLocal` y `variableGlobal`.

- Fuera de la función, solo se puede acceder a la variable global `variableGlobal`, ya que la variable local `variableLocal` solo existe dentro del ámbito de la función.

Aplicación en ambas sintaxis:

El concepto de ámbito de variables se aplica de la misma manera en ambas sintaxis de funciones en Dart: tradicional y flecha. La sintaxis utilizada para definir la función no afecta la accesibilidad de las variables dentro de su ámbito.

Ejemplo con sintaxis flecha:

```
var variableGlobal = "Valor global";

function mostrarVariableFlecha() => {
  var variableLocal = "Valor local";
  print(variableLocal); // Imprime "Valor local"
  print(variableGlobal); // Imprime "Valor global"
};

mostrarVariableFlecha();

// print(variableLocal); // Error: variableLocal no está definida
// fuera de la función
```

Consideraciones importantes:

- El ámbito de las variables es esencial para mantener el código organizado y evitar errores.
- Las variables locales solo deben utilizarse dentro de la función donde se declaran.
- Las variables globales deben utilizarse con cuidado, ya que pueden ser modificadas desde cualquier lugar del programa.

Programación Orientada a Objetos (POO) en Dart



La Programación Orientada a Objetos (POO) es un paradigma de programación fundamental en Dart. Permite organizar el código de manera modular y reutilizable, creando estructuras que modelan el mundo real.

Principios fundamentales de la Programación Orientada a Objetos (POO)

La Programación Orientada a Objetos (POO) es un paradigma de programación fundamental en el desarrollo de software moderno. Se basa en la creación de objetos, que son entidades encapsuladas que agrupan datos (atributos) y comportamiento (métodos).

Los cuatro pilares fundamentales de la POO son:

- **Abstracción:**
La abstracción es el proceso de ocultar los detalles de implementación de un objeto y exponer solo las interfaces necesarias para interactuar con él.
Permite enfocarse en el qué hace un objeto, sin preocuparse por el cómo lo hace.
Promueve la modularidad y la reutilización del código.
Ejemplo:
 - Una clase Coche puede abstraer los detalles de la implementación del motor, la transmisión y otros componentes

internos, exponiendo métodos como arrancar(), acelerar() y frenar().

- Encapsulamiento:

El encapsulamiento es el mecanismo de agrupar datos y métodos relacionados dentro de una clase, ocultando su implementación interna al resto del programa.

Protege los datos de modificaciones no deseadas y promueve la consistencia del estado del objeto.

Se implementa utilizando modificadores de acceso como public, private y protected.

Ejemplo:

- La clase Coche puede encapsular atributos privados como `_velocidadActual` y métodos privados como `_calcularVelocidad()`, exponiendo solo métodos públicos como `obtenerVelocidad()`.

- Herencia:

La herencia permite a una clase heredar atributos y métodos de otra clase padre.

Crea una jerarquía de clases donde las clases hijas pueden especializar el comportamiento de las clases base.

Promueve la reutilización del código y la flexibilidad en el diseño de software.

Ejemplo:

- Una clase Camioneta puede heredar de una clase base Vehículo, compartiendo atributos como marca, modelo y color, y agregando atributos específicos como capacidadDeCarga.

- Polimorfismo:

El polimorfismo es la capacidad de los objetos de responder al mismo mensaje de manera diferente.

Se basa en la sobreescritura de métodos en las clases hijas.

Permite crear código más flexible y adaptable.

Ejemplo:

- Las clases Coche, Camioneta y Motocicleta pueden heredar un método `dibujar()`, pero cada clase puede implementar su propia versión específica para dibujar su forma en la pantalla.

En resumen, estos cuatro principios fundamentales (abstracción, encapsulamiento, herencia y polimorfismo) son la base de la POO y permiten crear software modular, reutilizable, flexible y mantenible.

Conceptos fundamentales de la Programación Orientada a Objetos (POO)

Los cuatro principios fundamentales de la POO (abstracción, encapsulamiento, herencia y polimorfismo) sientan las bases para crear clases, que son los planos o plantillas a partir de los cuales se crean objetos.

Las clases definen las características y el comportamiento que comparten todos los objetos de la misma clase.

- Atributos: Representan las propiedades o características de los objetos, como su nombre, color, tamaño, etc.
- Métodos: Definen el comportamiento de los objetos, es decir, las acciones que pueden realizar, como encenderse, moverse, calcular algo, etc.

Los objetos son instancias de una clase específica. Poseen valores concretos para los atributos de la clase y pueden interactuar con otros objetos a través de sus métodos.

Los cuatro principios de la POO trabajan juntos para crear un sistema organizado y eficiente para la creación y uso de objetos:

- Abstracción: Permite ocultar la complejidad interna de las clases y objetos, exponiendo solo las interfaces necesarias para interactuar con ellos.
- Encapsulamiento: Protege los datos de los objetos y promueve la consistencia del estado.
- Herencia: Permite reutilizar código y crear jerarquías de clases con relaciones padre-hijo.
- Polimorfismo: Permite a los objetos de diferentes clases responder al mismo mensaje de manera diferente, aumentando la flexibilidad del código.

La combinación de estos principios permite crear software modular, reutilizable, flexible y mantenible.

Diferencias entre la POO en Dart y la POO en general

Si bien la POO en Dart comparte los principios fundamentales con la POO en otros lenguajes, existen algunas diferencias notables:

- **Sintaxis:**
Dart utiliza una sintaxis más concisa y expresiva para definir clases, objetos, métodos y herencia.
No se requieren palabras clave como `public`, `private` o `protected` para la modificación de accesibilidad, ya que se infiere por la ubicación de los miembros de la clase.
- **Mixins:**
Dart introduce el concepto de mixins, que permiten reutilizar código de comportamiento entre clases sin necesidad de herencia tradicional. Los mixins facilitan la composición de funcionalidades y la creación de clases más flexibles.
- **Funciones como ciudadanos de primera clase:**
En Dart, las funciones son ciudadanos de primera clase, lo que significa que pueden ser pasadas como argumentos, devueltas como valores de retorno y almacenadas en variables.
Esto permite una mayor flexibilidad y potencia en la programación orientada a objetos.
- **Tipado:**
Dart es un lenguaje de tipado estático, lo que significa que los tipos de datos de las variables y expresiones deben declararse explícitamente. Esto ayuda a detectar errores de tipo en tiempo de compilación y mejora la seguridad del código.
- **Anotaciones:**
Dart utiliza anotaciones para proporcionar metadatos adicionales a las clases, métodos y otros elementos del código.
Las anotaciones pueden ser utilizadas por herramientas de análisis estático, frameworks y bibliotecas para generar código o realizar otras tareas.
- **Librerías estándar:**
Dart incluye una librería estándar completa que proporciona una amplia gama de clases y funciones para el desarrollo de aplicaciones.

Esto facilita el trabajo con tareas comunes como la entrada/salida, el manejo de cadenas, las redes y la concurrencia.

En resumen, la POO en Dart conserva los principios fundamentales de la POO en general, pero introduce características y sintaxis modernas que la hacen más concisa, expresiva y flexible.

Ejemplo de creación de clase, objeto, atributos y métodos en Dart

Consideremos la creación de una clase Coche con atributos y métodos para representar un vehículo:

Creación de la clase

```
class Coche {  
    // Atributos (propiedades)  
  
    String marca;  
  
    String modelo;  
  
    int color; // 0: rojo, 1: azul, 2: verde  
  
    int velocidadActual = 0; // km/h  
  
    // Métodos (comportamiento)  
  
    void arrancar() {  
        print("El coche $marca $modelo está arrancando...");  
    }  
  
    void acelerar(int velocidad) {  
        if (velocidad > 0) {  
            velocidadActual += velocidad;  
  
            print("El coche $marca $modelo está acelerando a  
$velocidadActual km/h");  
        }  
    }  
}
```



```
} else {  
    print("No se puede acelerar a una velocidad negativa");  
}  
}  
  
void frenar(int velocidad) {  
    if (velocidad > 0) {  
        velocidadActual -= velocidad;  
        if (velocidadActual < 0) {  
            velocidadActual = 0;  
            print("El coche $marca $modelo se ha detenido");  
        } else {  
            print("El coche $marca $modelo está frenando a  
$velocidadActual km/h");  
        }  
    } else {  
        print("No se puede frenar a una velocidad negativa");  
    }  
}  
  
void mostrarInformacion() {  
    print("Información del coche:");  
    print("Marca: $marca");  
    print("Modelo: $modelo");  
    print("Color: ${getColorEnString(color)}");  
}
```

```
        print("Velocidad actual: $velocidadActual km/h");
    }

    String getColorEnString(int color) {
        switch (color) {
            case 0:
                return "Rojo";
            case 1:
                return "Azul";
            case 2:
                return "Verde";
            default:
                return "Color desconocido";
        }
    }
}
```

Este código define una clase llamada Coche que representa un vehículo y define:

- los atributos marca, modelo, color y velocidadActual.
- los métodos arrancar(), acelerar(), frenar() y mostrarInformacion().

Creación de un objeto a partir de la clase:

```
var miCoche = new Coche();

// Asignación de valores a los atributos
miCoche.marca = "Toyota";
miCoche.modelo = "Yaris";
miCoche.color = 1; // Azul

// Uso de los métodos
miCoche.arrancar();
miCoche.acelerar(20);
miCoche.frenar(10);
miCoche.mostrarInformacion();
```

Explicación:

A partir de la clase se crea el Objeto miCoche:

- Es una instancia de la clase Coche.
- Se le asignan valores a los atributos marca, modelo y color.
- Se utilizan los métodos arrancar(), acelerar(), frenar() y mostrarInformacion() para interactuar con el objeto.

Este es un ejemplo básico de creación de clases, objetos, atributos y métodos en Dart. La POO ofrece un sinnúmero de posibilidades para modelar objetos del mundo real y crear software modular, reutilizable y flexible.

Recursos adicionales

Genérico:

- Documentación oficial de Dart: <https://dart.dev/>
- Tutoriales de Dart: <https://dart.dev/tutorials>
- Ejemplos de código en Dart: <https://github.com/topics/dart-examples>
- Foros y comunidades de Dart: <https://dart.dev/community>

Funciones:

- Documentación oficial de Dart sobre funciones: <https://dart.dev/language/functions>
- Artículo sobre funciones flecha en Dart: <https://medium.com/@sgg2123/arrow-functions-in-js-ff54f558185b>
- Comparación entre funciones flecha y funciones tradicionales: <https://www.geeksforgeeks.org/what-is-the-arrow-function-and-how-to-create-it/>
- Tutoriales sobre funciones en Dart: <https://m.youtube.com/watch?v=W-AwQpWM4f0>

¡Enhorabuena por completar esta unidad didáctica sobre Funciones y Programación Orientada a Objetos en Dart!

Has revisado los conceptos fundamentales que te permitirán crear aplicaciones Dart más robustas, flexibles y organizadas.

Recuerda que la práctica es esencial para dominar estos conceptos. Te invito a que sigas explorando, experimentando y creando proyectos con Dart para consolidar tus conocimientos y convertirte en un desarrollador Dart más hábil.

¡Te deseo mucho éxito en tu viaje como desarrollador Dart!

¡Hasta la próxima!